

AVL Trees: Mastery Guide

Self-Balancing Binary Search Tree Optimization

1. What is an AVL Tree?

Invented in 1962 by Adelson-Velsky and Landis, the **AVL Tree** is the first self-balancing Binary Search Tree. It solves the "skewed tree" problem where a standard BST can degrade into a linked list with $O(n)$ performance.

The Golden Rule: An AVL tree maintains balance by ensuring that the height difference (Balance Factor) between the left and right subtrees of any node is at most 1.

2. The Balance Factor (BF)

The balance factor of a node is calculated as:

$$\text{BF}(\text{node}) = \text{Height}(\text{Right Subtree}) - \text{Height}(\text{Left Subtree})$$

- **BF = 0:** Perfectly balanced.
- **BF = 1 or -1:** Slightly heavy, but still balanced.
- **BF > 1 or < -1:** Unbalanced — requires a rotation.

3. The Four Rotation Cases

To restore balance, AVL trees use four types of rotations based on the path of the newly inserted node:

Case	Balance State	Requirement
Left-Left (LL)	Node is Left-Heavy, Child is Left-Heavy	Single Right Rotation
Right-Right (RR)	Node is Right-Heavy, Child is Right-Heavy	Single Left Rotation
Left-Right (LR)	Node is Left-Heavy, Child is Right-Heavy	Left Rotate Child, then Right Rotate Node
Right-Left (RL)	Node is Right-Heavy, Child is Left-Heavy	Right Rotate Child, then Left Rotate Node

4. Python Implementation: Rotations & Insertion

Here is how a Single Right Rotation is implemented to fix an LL case:

```
def rightRotate(y):
    x = y.left
    T2 = x.right
    # Perform rotation
    x.right = y
    y.left = T2
    # Update heights
    y.height = 1 + max(getHeight(y.left), getHeight(y.right))
    x.height = 1 + max(getHeight(x.left), getHeight(x.right))
    return x
```

This ensures the BST property is maintained while reducing the overall height of the subtree.

5. Time Complexity: Why $O(\log n)$?

Because the height of an AVL tree is always kept logarithmic relative to the number of nodes ($h \approx \log n$), operations are guaranteed to be fast.

Search: $O(\log n)$

Insert: $O(\log n)$

Delete: $O(\log n)$

In a perfect tree, $n = 2^{h+1} - 1$. Solving for h gives us the logarithmic relationship that makes AVL trees essential for high-performance databases.

Advanced Retracing & Recursion (Part 1)

Retracing is the process of moving from a newly inserted leaf back up to the root, checking each ancestor for balance. In recursion, this happens during the 'unwinding' of the stack.

Insight: A single insertion in an AVL tree requires at most **one** rotation (single or double) to restore balance globally. However, deletion may require rotations at multiple levels up to the root.

Visualizing LR and RL Rotations

A Left-Right (LR) rotation is essentially a "kink" in the tree. Because it is double-heavy in opposite directions, a single rotation cannot fix it. We first transform it into an LL case by rotating the child, and then resolve the LL case with a right rotation on the parent.

Advanced Retracing & Recursion (Part 2)

Retracing is the process of moving from a newly inserted leaf back up to the root, checking each ancestor for balance. In recursion, this happens during the 'unwinding' of the stack.

Insight: A single insertion in an AVL tree requires at most **one** rotation (single or double) to restore balance globally. However, deletion may require rotations at multiple levels up to the root.

Visualizing LR and RL Rotations

A Left-Right (LR) rotation is essentially a "kink" in the tree. Because it is double-heavy in opposite directions, a single rotation cannot fix it. We first transform it into an LL case by rotating the child, and then resolve the LL case with a right rotation on the parent.

Advanced Retracing & Recursion (Part 3)

Retracing is the process of moving from a newly inserted leaf back up to the root, checking each ancestor for balance. In recursion, this happens during the 'unwinding' of the stack.

Insight: A single insertion in an AVL tree requires at most **one** rotation (single or double) to restore balance globally. However, deletion may require rotations at multiple levels up to the root.

Visualizing LR and RL Rotations

A Left-Right (LR) rotation is essentially a "kink" in the tree. Because it is double-heavy in opposite directions, a single rotation cannot fix it. We first transform it into an LL case by rotating the child, and then resolve the LL case with a right rotation on the parent.

Advanced Retracing & Recursion (Part 4)

Retracing is the process of moving from a newly inserted leaf back up to the root, checking each ancestor for balance. In recursion, this happens during the 'unwinding' of the stack.

Insight: A single insertion in an AVL tree requires at most **one** rotation (single or double) to restore balance globally. However, deletion may require rotations at multiple levels up to the root.

Visualizing LR and RL Rotations

A Left-Right (LR) rotation is essentially a "kink" in the tree. Because it is double-heavy in opposite directions, a single rotation cannot fix it. We first transform it into an LL case by rotating the child, and then resolve the LL case with a right rotation on the parent.